

Configuración de parámetros de terminales TAS

El esquema de configuración se compone de dos tablas principales:

- PARAMETROS : define los parámetros disponibles y la información necesaria para interpretar y editar sus valores.
- PARAMETROSEXTERMINAL : almacena el valor configurado de cada parámetro para cada terminal.

Los campos CDATE , CUSER , MDATE y MUSER mantienen el esquema de auditoría actualmente utilizado por la aplicación.

Tabla PARAMETROS

Define los parámetros válidos que pueden ser utilizados para configurar las terminales.

Campo	Tipo	Obligatorio	Descripción
NOEMBARETARIO	VARCHAR(20)	Sí	Identificador único del parámetro. Es la clave primaria de la tabla. Ej.: ORIGEN, COBERTURAS_ADMITIDAS.
DESCRIPCION	VARCHAR(40)	Sí	Descripción funcional del parámetro y de su finalidad.

O N	0 0 0 C H A R)		
T I P O _ D A T O	V A R C H A R 2(2 0 C H A R)	Sí	Tipo de dato almacenado. Valores admitidos: STRING, NUMBER, BOOLEAN, JSON.
M U L T I P L E	C H A R (1 C H A R)	Sí	Indica si el parámetro admite múltiples valores. Valores posibles: S o N.
O R I G E N _ D A T O	V A R C H A R 2(1 0 0	No	Identifica el origen de los valores disponibles cuando éstos deben seleccionarse de un conjunto determinado. Ej.: COBERTURAS. Si es NULL, el valor no depende de un origen de datos.

S	C H A R)		
C D A T E	D A T E	No	Fecha de creación del registro.
C U S E R	V A R C H A R 2(3 0 C H A R)	No	Usuario que creó el registro.
M D A T E	D A T E	No	Fecha de última modificación del registro.
M U S E R	V A R C H A R 2(3 0 C H A	No	Usuario que realizó la última modificación.

	R)		
--	--------	--	--

Interpretación de los metadatos

Algunos ejemplos:

NOMBRE_PARA METRO	TIPO_DAT O	MULTIPL E	ORIGEN_D ATOS	Interpretación
OR IG EN	N U M B E R	N	O R I G E N E S - T A S	Un único identificador numérico seleccionado desde el catálogo de orígenes.
CO BE RT UR AS _A DM ITI DA S	N U M B E R	S	C O B E R T U R A S	Lista de identificadores numéricos correspondientes a coberturas médicas.
CO BE RT UR AS _E MI TE _C UP ON _P AG	N U M B E R	S	C O B E R T U R A S	Lista de coberturas para las cuales se permite emitir cupones de pago.

O				
ME NS AJ E_ BIE NV EN ID A	S T R I N G	N	N U L L	Texto de configuración libre.
CO NFI GU RA CI ON _E SP ECI AL	J S O N	N	N U L L	Configuración estructurada almacenada como JSON.

SQL

```

CREATE TABLE PARAMETROS (
    NOMBRE_PARAMETRO VARCHAR2(100 CHAR) NOT NULL,
    DESCRIPCION      VARCHAR2(4000 CHAR) NOT NULL,
    TIPO_DATO        VARCHAR2(20 CHAR) NOT NULL,
    MULTIPLE         CHAR(1 CHAR)      DEFAULT 'N' NOT NULL,
    ORIGEN_DATOS     VARCHAR2(100 CHAR),
    .
    CDATE            DATE,
    CUSER            VARCHAR2(30 CHAR),
    MDATE            DATE,
    MUSER            VARCHAR2(30 CHAR),
    .
    CONSTRAINT PK_PARAMETROS
        PRIMARY KEY (NOMBRE_PARAMETRO),
    .
    CONSTRAINT CK_PARAMETROS_TIPO_DATO
        CHECK (
            TIPO_DATO IN (
                'STRING',
                'NUMBER',
                'BOOLEAN'
            )
        )

```

```

    ),
.
    CONSTRAINT CK_PARAMETROS_MULTIPLE
        CHECK (MULTIPLE IN ('S', 'N'))
);
.

```

Tabla PARAMETROSXTERMINAL

Almacena los valores correspondientes a cada parámetro configurado para una terminal.

La combinación de `TERMINAL` y `NOMBRE_PARAMETRO` constituye la clave primaria, por lo que un mismo parámetro puede existir en múltiples terminales, pero solamente una vez para cada terminal.

C a m p o	Ti po	O b l i g a t o r i o	Descripción
T E R M I N A L	V A R C H A R 2(10 00 C H A R)	S í	Identificador de la terminal a la cual pertenece la configuración.
N O M B R E	V A R C	S í	Parámetro configurado. Referencia a PARAMETROS.NOMBRE_PA RAMETRO

R E - P A R A M E T R O	H A R 2(10 0 C H A R)		.
V A L O R	V A R C H A R 2(40 00 C H A R)	S í	Valor almacenado del parámetro. Su interpretación depende de TIPO_DATO y MULTIPLE definidos en PARAMETROS .
A C T I V O	C H A R(1 C H A R)	S í	Indica si la configuración debe aplicarse. Valores posibles: S o N . Permite deshabilitar temporalmente un valor sin eliminarlo.
C D A T E	D A T E	N o	Fecha de creación del registro.
C U S	V A R	N o	Usuario que creó el registro.

E R	C H A R 2(30 C H A R)		
M D A T E	D A T E	N o	Fecha de última modificación del registro.
M U S E R	V A R C H A R 2(30 C H A R)	N o	Usuario que realizó la última modificación.

Ejemplos de valores

TE R MI NA L	NOMBRE_PAR AMETRO	VA LO R	A C TI V O
TA CP OL II	ORIGEN	12	S
TA CP	COBERTURAS_ ADMITIDAS	[50, 72,9	S

OL I1		1]	
TA CP OL I1	COBERTURAS_ EMITE_CUPON_ PAGO	[50]	S
TA C2	COBERTURAS_ ADMITIDAS	[50, 72]	S
TA C2	MENSAJE_BIEN VENIDA	Bien veni do	N

SQL

```

CREATE TABLE PARAMETROSXTERMINAL (
    TERMINAL          VARCHAR2(1000 CHAR) NOT NULL,
    NOMBRE_PARAMETRO  VARCHAR2(100 CHAR)  NOT NULL,
    VALOR              VARCHAR2(4000 CHAR) NOT NULL,
    ACTIVO             CHAR(1 CHAR)        DEFAULT 'S' NOT NULL,
    .
    CDATE              DATE,
    CUSER              VARCHAR2(30 CHAR),
    MDATE              DATE,
    MUSER              VARCHAR2(30 CHAR),
    .
    CONSTRAINT PK_PARAMETROSXTERMINAL
        PRIMARY KEY (TERMINAL, NOMBRE_PARAMETRO),
    .
    CONSTRAINT FK_PXT_PARAMETRO
        FOREIGN KEY (NOMBRE_PARAMETRO)
        REFERENCES PARAMETROS (NOMBRE_PARAMETRO),
    .
    CONSTRAINT CK_PXT_ACTIVO
        CHECK (ACTIVO IN ('S', 'N'))
);
.

```

Consideraciones sobre VALOR

El campo VALOR se almacena como VARCHAR2(4000 CHAR) independientemente del tipo lógico del parámetro.

El tipo real se obtiene a partir de `PARAMETROS.TIPO_DATO` .

Ejemplos:

```
TIPO_DATO = NUMBER
VALOR = 12
```

.

```
TIPO_DATO = BOOLEAN
VALOR = true
```

.

```
TIPO_DATO = STRING
VALOR = Bienvenido
```

.

Cuando `MULTIPLE = 'S'` , el valor puede almacenarse como un array JSON:

```
[50, 72, 91]
```

.

Por ejemplo:

```
NOMBRE_PARAMETRO = COBERTURAS_ADMITIDAS
TIPO_DATO          = NUMBER
MULTIPLE           = S
ORIGEN_DATOS       = COBERTURAS
```

.

indica que el parámetro contiene una lista de identificadores numéricos que deben corresponder a elementos obtenidos desde el origen de datos `COBERTURAS` .

El campo `ORIGEN_DATOS` permite que la interfaz de configuración muestre los valores con su código y descripción en lugar de requerir que el usuario conozca los identificadores internos.

NOTA: ¡Si se va a usar ACTIVO para PARAMETROS_XTERMINAL, que se maneje como un campo más y no como auditoría automática.

Basic Image Generation

```
<Template data={{
  API_KEY_REF,
  MODEL: 'google/gemini-2.5-flash-image'
}}>
```

.

```
const openRouter = new OpenRouter({ apiKey: '{{API_KEY_REF}}' });
```

```

const result = await openRouter.chat.send({ model: '{{MODEL}}', messages: [{ role:
  'user', content: 'Generate a beautiful sunset over mountains with cinematic
lighting and reflections on a lake' }], modalities: ['image', 'text' ]});
const images = result.choices[0].message.images ?? [];
for (const [index, image] of images.entries()) {
  const imageUrl = image.image_url.url;
  console.log(`Generated image ${index + 1}: ${imageUrl.substring(0, 50)}...`);
}
.

```

Supported aspect ratios

The section following a long code sample must start below it without overlap.

Multi-page code sample

```

const generatedValue1 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 1) });
const generatedValue2 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 2) });
const generatedValue3 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 3) });
const generatedValue4 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 4) });
const generatedValue5 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 5) });
const generatedValue6 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 6) });
const generatedValue7 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 7) });
const generatedValue8 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 8) });
const generatedValue9 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 9) });
const generatedValue10 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 10) });
const generatedValue11 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 11) });
const generatedValue12 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 12) });
const generatedValue13 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 13) });
const generatedValue14 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 14) });
const generatedValue15 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 15) });
const generatedValue16 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 16) });
const generatedValue17 = await client.responses.create({ model: selectedModel,

```

[illegible]

```
const generatedValue41 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 41) });
const generatedValue42 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 42) });
const generatedValue43 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 43) });
const generatedValue44 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 44) });
const generatedValue45 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 45) });
const generatedValue46 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 46) });
const generatedValue47 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 47) });
const generatedValue48 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 48) });
const generatedValue49 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 49) });
const generatedValue50 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 50) });
const generatedValue51 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 51) });
const generatedValue52 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 52) });
const generatedValue53 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 53) });
const generatedValue54 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 54) });
const generatedValue55 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 55) });
.
```

Content after multi-page code

This heading and paragraph must appear after the final code line, never over it.

Section 1

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content

Section 2

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content

Section 3

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content

Section 4

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content